Task 2. Growth Performance Metrics

Project Overview:

This dashboard provides a structured, data-driven overview of the Growth Department's performance of the company across key subscription metrics. The department manages customer acquisition, retention and the health of the business across two segments: B2B and B2C.

Since the company operates on a monthly subscription model, sustainable success depends on maintaining active subscriptions, reducing churn, ensuring stable recurring revenue, and controlling operational costs. Therefore, the dashboard tracks revenue, subscription trends, retention performance, spending efficiency, and segment contribution.

It supports daily monitoring and strategic decision-making by offering clear insights into growth sustainability and overall financial performance.

Dataset Summary:

Created a sample dataset with total no. of tables: 5,

Customers Table

- Rows: 202, Columns: 5
- Columns: Customer_Id, Segment, signup_date, country, acquisition_channel

Subscriptions Table

- Rows: 200, Columns: 7
- Columns: subscription_id, customer_id, plan_type, start_date, end_date, monthly_price, status

Revenue Table

- Rows: 1,966, Columns: 5
- Columns: date, customer_id, segment, revenue_type, amount

Marketing Spend Table

- Rows: 96, Columns: 4
- Columns: date, segment, channel, spend.

Funnel Table

- Rows: 24, Columns: 6
- Columns: date, segment, visitors, leads, trials, paid_conversions

KPI Selection:

1. Revenue Compared to Previous Month: To monitor financial performance and detect short-term increases or declines in recurring revenue.

DAX used instead of static calculations.

- **Previous Month Revenue**

```
Previous Month Revenue =
CALCULATE(
    [LatestKPI],
    PARALLELPERIOD('Calendar'[Date],-1,MONTH)
)
```

- **Latest KPI**

```
LatestKPI = CALCULATE(
    SUM(revenue[amount]),
    DATESBETWEEN(
        'Calendar'[Date],
        [MinDate],
        [MaxDate]
            )
)
```

2. Active Subscriptions: This KPI was selected to measure the size of the revenue-generating customer base and evaluate acquisition and retention effectiveness.

- **Active Subscriptions**

```
Active Subscriptions =
VAR SnapshotDate = MAX('Calendar'[Date])
RETURN
CALCULATE(
    COUNTROWS(subscriptions),
    subscriptions[start_date] <= SnapshotDate,
    ISBLANK(subscriptions[end_date])
        || subscriptions[end_date] > SnapshotDate
)
```

- **Active Subs Previous Month**

```
Active Subs Previous Month =
CALCULATE (
    [Active Subscriptions],
    DATEADD('Calendar'[Date], -1, MONTH))
```

3. Churned Subscriptions: To track customer cancellations and assess retention risk that directly impacts future revenue.

- **Churned Subscriptions**

```
Churned Subscriptions =
VAR _Churn =
   CALCULATE(
      COUNTROWS(Subscriptions),
      FILTER(
         subscriptions,
         NOT ISBLANK(subscriptions[end_date])
            &&
         subscriptions[end_date] >= [MinDate]
            &&
         subscriptions[end_date] <= [MaxDate]
      )
   )
RETURN
COALESCE(_Churn, 0)
```

4. Spend Compared to Previous Month: To evaluate cost efficiency and ensure that growth investments remain financially sustainable.

- **Latest Spends**

```
LatestSpends =
COALESCE(
   CALCULATE(
      SUM(marketing_spend[spend]),
      DATESBETWEEN(
         'Calendar'[Date],
         [MinDate],
         [MaxDate]  )   ), 0)
```

- **Previous Month Spends**

```
Previous Month Spends =
CALCULATE (
   [LatestSpends],
   PARALLELPERIOD('Calendar'[Date], -1,MONTH)
```

5. Revenue by Month and Segment (B2B vs B2C): To compare business unit performance and support strategic resource allocation between segments.

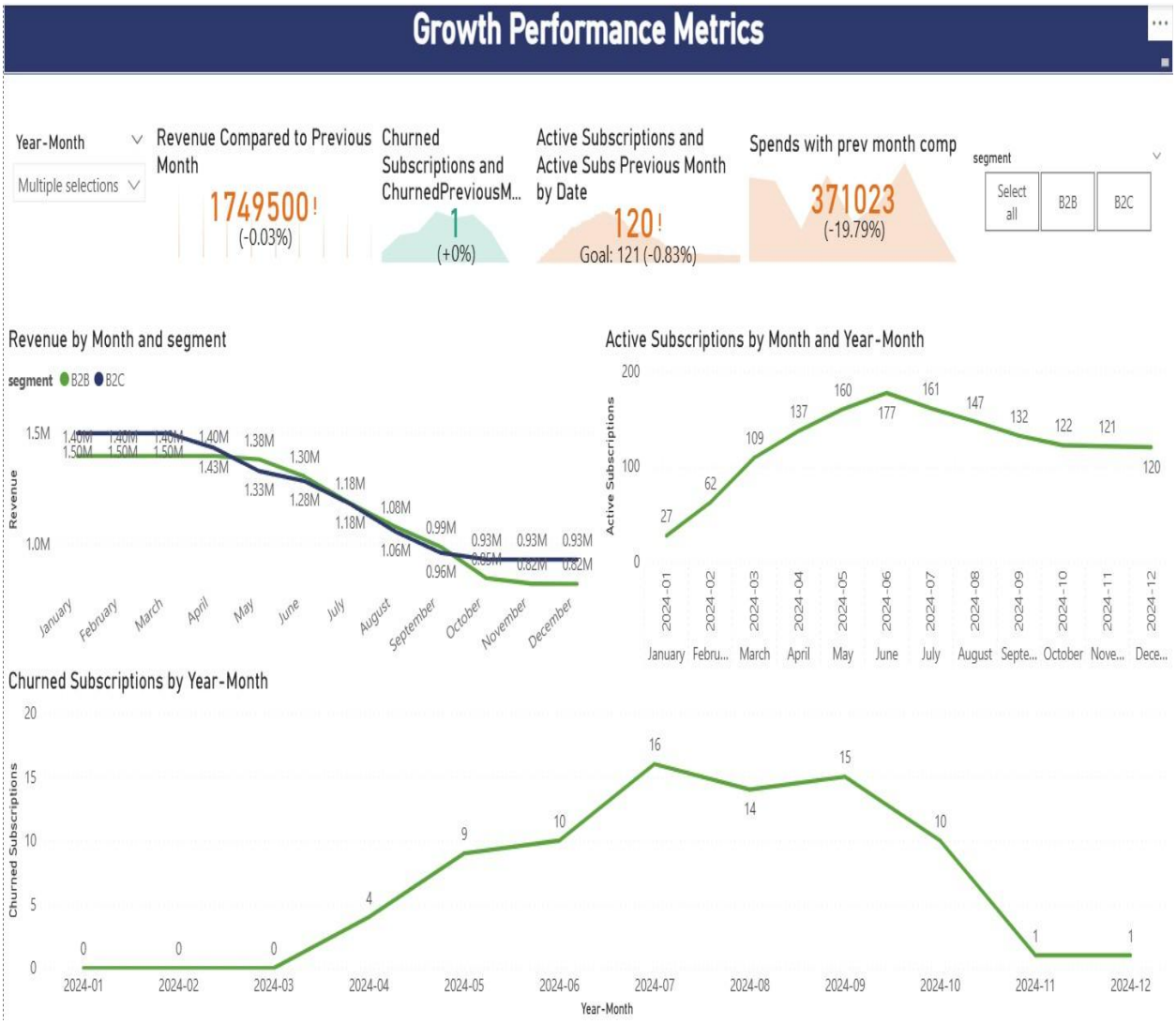
- **Month**

```
Month = FORMAT(revenue[date],"MMMM")
```

Business Impact:

The dashboard shows that revenue remains stable, but there were customer losses in the middle of the year. It helps the team take early action to improve retention and control costs. Insights from B2B and B2C performance support better decisions, encouraging steady and sustainable growth.

Dashboard Layout



Conclusion

The Growth Performance Metrics Dashboard offers a clear and structured view of subscription performance, including revenue, active subscriptions, churn, spend, and segment contribution. By focusing on financial stability and retention trends, it supports informed decision-making and promotes sustainable growth through continuous performance monitoring and strategic insight.

Attachments:

Growth_Dashboard.pbix

Sample.xlsx

SQL

1.How many applications were submitted over the observed period?

Row	total_applications
1	15331

2. What is the average number of applications submitted per week?

Row	avg_applications...
1	2190.14

3. How many applications were submitted per user role?

Row	user_role	applications_count
1	["admin"]	12723
2	["driver"]	2707
3	Unknown	1247
4	["admin","analyst"]	20
5	["analyst"]	15
6	["admin","driver"]	12

4. How many applications were submitted based on company size?

Company sizes are categorized as:

- Small: Fleet up to 50 cars
- Mid: Fleet from 50 to 150 cars
- Large: Fleet over 150 cars

Row	company_size	applications_count
1	Mid	8767
2	Small	3902
3	Large	2640
4	Unknown	22

Attachment: https://console.cloud.google.com/bigquery?ws=!1m7!1m6!12m5!1m3!1sfinn-case-study-487820!2seurope-west3!3s0d14c30a-2db6-4ff2-a946-ba05178e171a!2e1

CODE:

```
-- Sessionize Requests into Applications

WITH cleaned_requests AS (
    SELECT
        mapped_user_id,
        TIMESTAMP(timestamp) AS request_time
    FROM `finn-case-study-487820.finn_case.requests_log`
),

ordered_requests AS (
    SELECT
        mapped_user_id,
        request_time,
        LAG(request_time) OVER (
            PARTITION BY mapped_user_id
            ORDER BY request_time
        ) AS previous_request_time
    FROM cleaned_requests
),

flagged_requests AS (
    SELECT
        mapped_user_id,
        request_time,
        CASE
            WHEN previous_request_time IS NULL THEN 1
            WHEN TIMESTAMP_DIFF(request_time, previous_request_time, SECOND) > 600 THEN 1
            ELSE 0
        END AS new_application_flag
    FROM ordered_requests
),
```

```
applications AS (
    SELECT
        mapped_user_id,
        request_time,
        SUM(new_application_flag) OVER (
            PARTITION BY mapped_user_id
            ORDER BY request_time
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
        ) AS application_id
    FROM flagged_requests
),

-- Distinct applications with start time
distinct_applications AS (
    SELECT
        mapped_user_id,
        application_id,
        MIN(request_time) AS application_time
    FROM applications
    GROUP BY mapped_user_id, application_id
),

-- 1. Total Applications

total_applications AS (
    SELECT
        COUNT(*) AS total_applications
    FROM distinct_applications
),

-- 2. Weekly Applications & Average
```

```
weekly_application_counts AS (
  SELECT
    DATE_TRUNC(DATE(application_time), WEEK(MONDAY)) AS week_start,
    COUNT(*) AS applications_per_week
  FROM distinct_applications
  GROUP BY week_start
),

average_weekly_applications AS (
  SELECT
    ROUND(AVG(applications_per_week), 2) AS avg_applications_per_week
  FROM weekly_application_counts
),

-- 3. Applications per User Role

applications_by_role AS (
  SELECT
    COALESCE(ur.roles, 'Unknown') AS user_role,
    COUNT(*) AS applications_count
  FROM distinct_applications da
  LEFT JOIN `finn-case-study-487820.finn_case.user_roles` ur
    ON da.mapped_user_id = ur.user_id
  GROUP BY user_role
),

-- 4. Applications by Company Size

applications_by_company_size AS (
  SELECT
    CASE
      WHEN uc.company_fleet_size <= 50 THEN 'Small'
```

```
 WHEN uc.company_fleet_size BETWEEN 51 AND 150 THEN 'Mid'
          WHEN uc.company_fleet_size > 150 THEN 'Large'
          ELSE 'Unknown'
      END AS company_size,
      COUNT(*) AS applications_count
   FROM distinct_applications da
   LEFT JOIN `finn-case-study-487820.finn_case.user_company` uc
      ON da.mapped_user_id = uc.user_id
   GROUP BY company_size
)

-- Final Select

-- 1. Total Applications
--SELECT * FROM total_applications;

--2. Average number of applications submitted per week
--SELECT * FROM average_weekly_applications;

--3. Applications submitted per user role
--SELECT * FROM applications_by_role ORDER BY applications_count DESC;

--4. Applications submitted based on company size
SELECT * FROM applications_by_company_size ORDER BY applications_count DESC;
```